

Plan de Trabajo — Reparto de Tareas (Tú + Diego)

1. Herramienta de calidad de código: qué usar y por qué

Recomendación: SonarCloud, no SonarQube self-hosted.

SonarQube "de verdad" necesita un servidor propio corriendo 24/7 (Docker, base de datos, mantenimiento). Para un equipo de 2 personas es sobrecarga que no necesitáis. **SonarCloud** es el mismo motor de análisis pero como SaaS de Sonar, se integra directo con GitHub Actions (que ya usáis), es gratis para repos y detecta exactamente el tipo de cosas que encontré en la auditoría: código duplicado (los bloques de filtros repetidos en `main.py`), los dos `comparar_productos`), variables/atributos inexistentes, complejidad ciclomática alta, code smells.

Lo que SonarCloud NO os va a pillar (importante saberlo, para no confiar ciegamente): el bug de `Producto.precio_anterior` en `newsletter_semanal.py` es un `AttributeError` de SQLAlchemy en tiempo de ejecución sobre un atributo dinámico — un analizador estático de Python normalmente no lo detecta porque no hace introspección de modelos ORM. Eso solo lo pilla un **test de integración real** que ejecute la query contra una BD de prueba. Por eso el plan de abajo incluye tests, no solo linters.

Stack recomendado (todo gratis/barato, encaja con lo que ya tenéis):

Capa	Herramienta	Qué hace
Análisis estático global	SonarCloud	Bugs, code smells, duplicación, cobertura de tests centralizada
Python lint/format	Ruff	Sustituye flake8+black+isort en una sola pasada, muy rápido
Python tests	pytest + <code>pytest-cov</code>	Tests de endpoints reales contra una BD de test (SQLite o Postgres de test) — esto es lo que os habría pillado el bug de <code>Producto.precio</code>
Seguridad Python	Bandit	Detecta patrones inseguros (ya cubristeis bien lo básico, pero es gratis tenerlo en CI)
TypeScript/React lint	ESLint (ya lo tenéis configurado) + activar <code>strict: true</code> en <code>tsconfig.json</code> si no lo está	
Frontend tests	Vitest o Jest + React Testing Library, empezar por componentes críticos (<code>Catalog.tsx</code> , <code>ProductCard.tsx</code>)	

Integración: un solo workflow nuevo `.github/workflows/quality.yml` que corra en cada Pull Request (no en cada push a main) con `ruff check`, `pytest --cov`, `eslint`, y el scanner de SonarCloud. Que sea **obligatorio pasar antes de mergear** (protección de rama en GitHub: Settings → Branches → Require status checks).

Esto es tarea de **una sola persona, una sola vez** — no la dividáis, porque tocar `.github/workflows/` y configs de raíz a la vez desde dos ramas es la receta perfecta para conflictos de merge en el primer día. Que la haga quien tenga más soltura con CI/CD, y el otro revisa el PR.

2. Qué es lo más urgente (recordatorio priorizado)

Por impacto en negocio, en este orden:

`newsletter_semanal.py` + `retargeting_vistas.py` rotos → dinero/usuarios que ahora mismo no estáis recuperando, cero coste de "feature nueva", solo arreglar.

`sitemap.ts` roto → sin esto, todo el trabajo de SEO futuro pierde eficacia porque Google no descubre las fichas.

Setup de CI/tests (arriba) → para que lo de los puntos 1-2 no vuelva a pasar sin que nadie se entere. Todo lo demás del backlog de la auditoría.

3. La regla de oro para no pisaros

El 90% de los conflictos de este proyecto van a venir de un único archivo: `backend/main.py`. Es un monolito de +900 líneas donde vive todo: productos, auth, favoritos, social, newsletter... Si los dos tocáis `main.py` en paralelo en ramas distintas, vais a tener merge conflicts constantes.

Reglas prácticas:

Un archivo grande compartido (`main.py`, `models.py`, `schemas.py`) = **una persona a la vez**. Antes de empezar una tarea que toque uno de estos, avisa en el chat del equipo "voy a tocar main.py ahora", termina, mergea, y solo entonces el otro empieza su parte ahí.

Todo lo demás, dividido por carpeta, así podéis trabajar en paralelo sin ni preguntar:

`frontend/src/**` → una persona

`backend/ingestores/`, `backend/scripts/`, CRONs sueltos → la otra persona

Nadie más toca `main.py`/`models.py`/`schemas.py` salvo que sea su bloque asignado esa semana.

Ramas cortas y PRs pequeños. Nada de una rama "feature/mejoras" que viva 3 semanas — eso es lo que garantiza conflictos gigantes al final. Un PR = una tarea de la lista de abajo, mergeado en 1-2 días máximo.

`git pull` **antes de empezar cualquier tarea nueva**, siempre.

4. Reparto por semanas — dos tracks en paralelo

TRACK A — "Backend / Datos / Fiabilidad" (sugerido: Diego, o quien conozca mejor los ingestores)

Archivos propios, sin solapamiento con Track B salvo lo indicado:

Semana 1

☐ Arreglar `backend/newsletter_semanal.py` (migrar a `Oferta.precio`, copiando patrón de `orquestador.py`)

- ☐ Arreglar `backend/retargeting_vistas.py` (mismo fix)
- ☐ Añadir alerta Telegram cuando cualquier CRON falle (nueva función pequeña en `services/`, o reutilizar `send_telegram_deals.py` como referencia de cómo pegan a la API de Telegram)
- ☐ Setup de pytest básico: 2-3 tests de integración que habrían pillado el bug (`test_newsletter_query_no_rompe`, etc.) — esto sirve de "prueba de que el bug está arreglado y no puede volver"

Semana 2 (⚠ único momento en que este track toca `main.py` — avisar antes)

- ☐ Limpiar `main.py`: quitar el bloque de filtros duplicado en `obtener_productos`, quitar el segundo `comparar_productos` repetido
- ☐ Decidir con Track B qué hacer con `/api/click/{id}` (borrar la llamada del frontend o implementar el endpoint) — es la única tarea que requiere coordinación explícita entre los dos

Semana 3-4

- ☐ Endpoint de reseñas de sabor: `POST/GET /api/resenas` en `main.py` + schema nuevo en `schemas.py` (avisar antes de tocar estos dos archivos)
- ☐ Endpoint `GET /api/stacks/{id}` y añadir `stacks` a `PerfilResponse`

Mes 2+

- ☐ Monitor de scrapers: script que valide que cada ingestor devolvió >0 productos, si no, avisa por Telegram
- ☐ Rate limiting en `/api/login`, `/api/registro`, `/api/newsletter/subscribe` (una sola pasada en `main.py`, avisar antes)
- ☐ Refactor de paginación SQL real en `/api/productos` (tarea grande, aislar en su propia rama, comunicar bien porque toca el corazón de `main.py`)

TRACK B — "Frontend / SEO / Producto visible" (sugerido: tú)

Archivos propios, casi todo dentro de `frontend/src/`, sin tocar backend salvo consumo de endpoints ya existentes:

Semana 1

- ☐ Arreglar `frontend/src/app/sitemap.ts` (paginación real en bloques de 200, corregir `data.items` → `data.productos`)
- ☐ Revisar `AuthContext`/almacenamiento del JWT — evaluar mover de `localStorage` a cookie httpOnly (esto puede requerir un pequeño cambio en backend de `/api/login` para setear cookie — coordinar con Track A si se decide hacer, si no, dejarlo para más adelante)

Semana 2

- ☐ `producto/[slug]/page.tsx`: cambiar `cache: 'no-store'` por `next: { revalidate: 3600 }`, añadir `canonical` explícito por producto
- ☐ Enriquecer JSON-LD con `offers` (array) en vez de un único `lowPrice`/`highPrice`
- ☐ Limpiar `TrackedAffiliateLink.tsx` según lo que se decida con Track A sobre `/api/click`

Semana 3-4

- ☐ Frontend del histórico de precios: gráfico simple (Chart.js o Recharts, ya está listado como librería disponible) consumiendo `historial_precios` que ya devuelve la API
- ☐ UI de reseñas de sabor en la ficha de producto (una vez Track A tenga el endpoint listo — si no está listo aún, dejar el componente con datos mock y conectar después, así no bloqueas tu semana esperando al otro)
- ☐ UI de stacks en el perfil público

Mes 2+

- ☐ Migrar categorías/marcas más buscadas a rutas SSR/ISR dedicadas (`/proteinas`, `/marcas/hsn`) en vez de solo query params
- ☐ Primeras 2-4 páginas de contenido editorial (guías de compra) para SEO long-tail

5. Sincronización mínima necesaria

No hace falta un Jira complejo para dos personas, pero sí un mínimo:

Un board simple (GitHub Projects, que es gratis y ya está integrado con vuestros PRs) con columnas: `To Do / Doing / Review / Done`, con las tareas de arriba ya cargadas.

Un mensaje corto cada vez que alguien vaya a tocar `main.py`, `models.py` o `schemas.py` — es la única regla que de verdad hace falta cumplir a rajatabla, todo lo demás es paralelizable sin fricción.

Sync rápido semanal (15 min): qué se ha mergeado, qué bloquea a quién (sobre todo las dos tareas de reseñas/stacks, donde frontend depende de que backend termine el endpoint primero).

Con esto deberíais poder ir en paralelo sin bloquearos, y el track A tiene los fixes de mayor impacto económico resueltos ya en la primera semana.